

# 1. Мои данные

Козенко Дмитрий; группа Б08; студенческий номер st117373.

## 2. Полное название задачи

**Задача китайского почтальёна на взвешенном неориентированном графе:** найти минимальную суммарную длину замкнутого обхода, при котором каждое ребро пройдено хотя бы один раз (при допущении, что стоимость повторного прохода по ребру равна длине этого ребра, а дублирование допускается по кратчайшим путям между вершинами).

## 3. Теоретическое описание задачи

Дан связный неориентированный граф  $G = (V, E)$  с положительными весами на рёбрах. Требуется найти *замкнутый* маршрут минимальной суммарной длины такой, что каждое ребро хотя бы раз входит в маршрут (допускаются повторы рёбер и вершин). Эту постановку называют задачей китайского почтальёна (CPR).

Граф является эйлеровым тогда и только тогда, когда он связан и все степени вершин чётны. Если граф уже эйлеров, оптимум равен сумме длин всех рёбер (каждое один раз).

Если вершины нечётной степени есть, чтобы получить эйлеровость, нужно «дублировать» проезд по некоторым путям; суммарную дополнительную длину минимизируют, жёстко сопоставив нечётным вершинам пары и для каждой пары платя за кратчайший путь между её концами. Число нечётных вершин всегда чётно; при малом числе таких вершин  $k$  задачу минимального идеального паросочетания на полном графе с весами кратчайших путей удобно решать методом динамического программирования по подмножествам вершин (битмаски).

## 4. Теоретическое описание решения

**Чтение и проверки.** По входному файлу строят список рёбер, проверяя границы вершин  $1 \dots n$ , положительные веса, комментарии и пустые строки игнорируются.

**Подсумм базовых рёбер.** Одновременно с загрузкой в матрице смежности фиксируют минимальные длины на ребрах (при возможных дубликатах рёбер) и суммируют вес каждого уникально заданного ребра один раз как «базовую» сумму сети.

**Нечётные степени.** Вершины нечётной степени извлекаются в упорядоченный массив. Если массив пуст, программа сообщает сумму только базовых рёбер — это уже ответ без дублирований.

**Кратчайшие пути.** Алгоритм Флойда–Уоршелла восстанавливает матрицу кратчайших расстояний между всеми городами для дальнейшего подбора недорогих добавок.

**Минимальное паросочетание среди нечётных вершин.** Для  $k$  нечётных вершин к перебору состояний  $2^k$  считают минимальную стоимость разбиения попарным соединением: рекуррентно добавляют пары необработанных нечётных вершин, штрафляя за их кратчайшее расстояние.

**Вывод.** Печатаются таблицы (в том числе матрица кратчайших расстояний при  $n \leq 25$ ) и строка с финальной суммой: базовая сумма рёбер плюс оптимальная дополнительная длина путей.

## 5. Программа — листинг main.cpp

Код включает вспомогательные заголовки `../common/io_ru.hpp`, `../common/table_ru.hpp` (ввод из файла, табличный вывод). Ниже — основной файл (при сборке отчёта Unicode-символы псевдографики заменены на ASCII, чтобы сборка через `pdflatex` проходила стабильно):

```
#include "../common/io_ru.hpp"
#include "../common/table_ru.hpp"

#include <algorithm>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <vector>

using namespace std;

namespace {

constexpr int kInf = 1'000'000'000;

std::string strip(std::string s) {
    while (!s.empty() && isspace(static_cast<unsigned char>(s.front()))) s.erase(0, 1);
    while (!s.empty() && isspace(static_cast<unsigned char>(s.back()))) s.pop_back();
    return s;
}

bool skip_line(const std::string& ln) {
    const std::string t = strip(ln);
    return t.empty() || t.front() == '#';
}

bool parse_triple(const std::string& ln, int& a, int& b, int& c) {
    std::istringstream is(strip(ln));
    return static_cast<bool>(is >> a >> b >> c);
}

bool load_input(std::istream& in, int& line_no_anchor, int& n, int& m,
               vector<tuple<int, int, int>>& edges, std::string& err) {
    vector<string> kept;
    string line;
    int raw = 0;
    while (getline(in, line)) {
        ++raw;
        if (skip_line(line)) continue;
        kept.push_back(strip(line));
    }
    if (kept.empty()) {
        err = "во входном файле нет ни одной значимой строки (учитываются строки после удаления комментариев #)";
        line_no_anchor = 0;
        return false;
    }
    size_t idx = 0;
    {
        std::istringstream iss(kept[idx]);
        if (!(iss >> n >> m)) {
            err = "первая строка с данными: ожидаются два целых числа n m (города и рёбра)";
            line_no_anchor = static_cast<int>(idx + 1);
            return false;
        }
    }
}
```

```

    }
    idx++;
    if (n < 1 || m < 0) {
        err = "нужно n>=1 и m>=0";
        line_no_anchor = idx;
        return false;
    }
    if (kept.size() - idx < static_cast<size_t>(m)) {
        err = "недостаточно строк после «n m»: ожидалось " + std::to_string(m) +
            " описаний рёбер вида «u v w»";
        line_no_anchor = idx + 1;
        return false;
    }
    edges.clear();
    for (int e = 0; e < m; ++e) {
        int u, v, w;
        if (!parse_triple(kept[idx + static_cast<size_t>(e)], u, v, w)) {
            err = "строка #" + std::to_string(static_cast<int>(idx) + e + 1) +
                " среди блока рёбер: ожидается «город_u город_v длина»";
            line_no_anchor = static_cast<int>(idx) + e + 1;
            return false;
        }
        edges.emplace_back(u, v, w);
    }
    return true;
}

void floydWarshall(int n, vector<vector<int>>& dist) {
    for (int k = 1; k <= n; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (dist[i][k] >= kInf) continue;
            for (int j = 1; j <= n; ++j) {
                if (dist[k][j] >= kInf) continue;
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}

vector<int> oddVertices(int n, const vector<int>& degree) {
    vector<int> out;
    out.reserve(static_cast<size_t>(n));
    for (int v = 1; v <= n; ++v)
        if (degree[v] & 1) out.push_back(v);
    return out;
}

bool oddPairsAllFinite(const vector<int>& odd, const vector<vector<int>>& dist) {
    for (size_t i = 0; i < odd.size(); ++i)
        for (size_t j = i + 1; j < odd.size(); ++j)
            if (dist[odd[i]][odd[j]] >= kInf) return false;
    return true;
}

int minOddPairingCost(const vector<int>& odd, const vector<vector<int>>& dist) {
    const int k = static_cast<int>(odd.size());
    const int full = 1 << k;
    vector<int> dp(full, kInf);
    dp[0] = 0;

    for (int mask = 0; mask < full; ++mask) {
        if (dp[mask] == kInf) continue;

        int i0 = -1;
        for (int i = 0; i < k; ++i) {
            if (!(mask & (1 << i))) {
                i0 = i;
                break;
            }
        }
        if (i0 == -1) continue;

        for (int j = i0 + 1; j < k; ++j) {

```

```

        if (mask & (1 << j)) continue;
        int next = mask | (1 << i0) | (1 << j);
        int w = dist[odd[i0]][odd[j]];
        dp[next] = min(dp[next], dp[mask] + w);
    }
}
return dp[full - 1];
}

void print_input_edges_table(const vector<tuple<int, int, int>>& edges) {
    vector<vector<string>> e_rows;
    e_rows.reserve(edges.size());
    for (size_t i = 0; i < edges.size(); ++i) {
        const auto& [u, v, w] = edges[i];
        e_rows.push_back({
            std::to_string(static_cast<int>(i) + 1),
            std::to_string(u),
            std::to_string(v),
            std::to_string(w),
        });
    }
    table_ru::print_table(std::cout, "Входные рёбра (как записаны во входном файле)",
        {"Номер", "u", "v", "Длина"}, e_rows);
}

void maybe_print_full_fw(int n, const vector<vector<int>>& dist, int sentinel) {
    constexpr int kMaxPrint = 25;
    if (n <= kMaxPrint) {
        std::vector<std::string> colh, rowh;
        for (int i = 1; i <= n; ++i) {
            std::string s = std::string("Г") + std::to_string(i);
            colh.push_back(s);
            rowh.push_back(s);
        }
        vector<vector<int>> sub(static_cast<size_t>(n));
        for (int i = 1; i <= n; ++i) {
            sub[static_cast<size_t>(i - 1)].assign(dist[i].begin() + 1, dist[i].begin() + n + 1);
        }
        table_ru::matrix_int_corner(std::cout, "Матрица кратчайших расстояний после Флойда-Уоршелла", sub,
            colh, rowh, sentinel);
    } else {
        cout << "\nПолная матрица кратчайших расстояний не печатается (n превышает порог показа 25, текущее n="
            << n << ").\n";
    }
}

vector<vector<string>> odd_pair_combo_table(const vector<int>& odd,
    const vector<vector<int>>& dist) {
    vector<vector<string>> rows;
    const size_t k = odd.size();
    if (k < 2) return rows;
    for (size_t i = 0; i < k; ++i)
        for (size_t j = i + 1; j < k; ++j) {
            rows.push_back({"{" + std::to_string(odd[i]) + ", " + std::to_string(odd[j]) + "}",
                std::to_string(dist[odd[i]][odd[j]])});
        }
    return rows;
}

} // namespace

int main(int argc, char** argv) {
    ifstream fin;
    if (!io_ru::open_only_read_strict(argc, argv, fin, cerr)) return 2;

    int n = 0, m = 0;
    vector<tuple<int, int, int>> edges;
    int line_anchor = 0;
    std::string err;

    auto fail_parse = [&](const std::string& e) -> int {
        cerr << "Файл «" << argv[1] << "» повреждён или некорректен: ";
        cerr << e;
    };

```

```

        if (line_anchor > 0) cerr << " (локальный номер блока данных: #" << line_anchor << ')';
        cerr << '\n';
        return 1;
    };

    if (!load_input(fin, line_anchor, n, m, edges, err)) return fail_parse(err);

    vector<vector<int>>> dist(n + 1, vector<int>(n + 1, kInf));
    vector<int> degree(n + 1, 0);
    int sum_edges = 0;

    for (int v = 1; v <= n; ++v) dist[v][v] = 0;

    for (const auto& [u_in, v_in, w_in] : edges) {
        int u = u_in, v = v_in, w = w_in;
        if (u < 1 || u > n || v < 1 || v > n || w <= 0) {
            err = "неверные данные ребра («" + std::to_string(u) + ">, «" +
                std::to_string(v) + ">, «" + std::to_string(w) + ">");
            cerr << err << ": номера городов должны быть 1.." << n << ", длина > 0\n";
            return 1;
        }
        sum_edges += w;
        dist[u][v] = min(dist[u][v], w);
        dist[v][u] = min(dist[v][u], w);
        degree[u]++;
        degree[v]++;
    }

    print_input_edges_table(edges);

    vector<int> odd = oddVertices(n, degree);

    if (odd.empty()) {
        table_ru::print_table(std::cout, "Итог",
            {"Показатель", "Число"},
            {"Базовая сумма рёбер (один раз по каждому рёберному отрезку)",
                std::to_string(sum_edges)});
        cout << "\nМинимальная длина маршрута почтальона (Euler): " << sum_edges << '\n';
        return 0;
    }

    floydWarshall(n, dist);
    maybe_print_full_fw(n, dist, kInf);

    if (!oddPairsAllFinite(odd, dist)) {
        table_ru::print_table(
            std::cout, "Результат - граф между нечётными вершинами не связан", {"Причина"},
            {"Некоторые пары вершин нечётной степени недостижимы по ограничению кратчайших расстояний "
                "(inf) - задача имеет недопустимо бесконечное дополнение."});
        cout << "\nМинимальная длина маршрута почтальона: недостижима (отсутствует конечное дополнение).\n";
        return 0;
    }

    vector<vector<string>>> pair_cost_rows = odd_pair_combo_table(odd, dist);
    // if (!pair_cost_rows.empty())
    //     table_ru::print_table(std::cout, "Попарная стоимость «соединить» нечётные вершины (кратчайший путь)", {"Пара", "Cost"},
    //         pair_cost_rows);

    const int extra = minOddPairingCost(odd, dist);

    table_ru::print_table(std::cout, "Числовой итог",
        {"Показатель", "Значение"},
        {"Доп. длина дублируемых проездов (минимум)", std::to_string(extra)},
        {"Базовая сумма рёбер", std::to_string(sum_edges)},
        {"Итоговая минимальная длина", std::to_string(sum_edges + extra)});

    cout << "\nМинимальная длина маршрута почтальона: " << (sum_edges + extra) << '\n';
    return 0;
}

```

## 6. Комментарии к коду

`floydWarshall` — классический трёхвложенный цикл расширения путей с «бесконечностью» `kInf`. `oddVertices` — подсчёт степени по инцидентным рёбрам и сбор индексов нечётных. `minOddPairingCost` — DP по битмаскам: из состояния с двумя непокрытыми нечётными вершинами минимально оплачивается объединённая пара. `load_input` — единственная точка разбора формата с русскими сообщениями ошибок для пользователя.

## 7. Разбор входного файла `sample.txt`

Строки файла следующие (первая — число вершин и рёбер, далее рёбра):

```
5 4
1 2 3
2 3 100
3 4 10
4 5 1
```

Граф содержит 5 городов и 4 ребро: тракт можно мысленно представить как путь цепью  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ , где длины 3, 100, 10, 1 соответственно между соседними в цепочке. Степени: вершины 2 и 4 имеют степени 2 каждая, вершины 1, 5 — степень 1 (концы), вершины 3 — степень 2. Нечётными остаются именно два конца трактового сегмента (1 и 5), то есть задача добавки сводится к одной возможной паре (1, 5) по кратчайшему пути  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$  длиной  $3+100+10+1 = 114$ , при этом базовая сумма всех имёнованных отрезков равна 114 же. Поэтому ожидаемый финальный результат — удвоенная сумма содержательных рёбер  $114+114 = 228$ : почтальон проходит каждое ребро один раз базово и оплачивает ещё раз тот же путь между нечётными концами. Программа в разделах вывода фиксирует таблицы и числовые итог согласованно этой логике.

## 8. Пример вывода программы при запуске на `sample.txt`

Ниже — перехваченный стандартный вывод (управляющие ANSI-последовательности удалены скриптом сборки при необходимости). В выводе есть таблица входных рёбер, при малых  $n$  — матрица кратчайших расстояний после Флойда, блок «Числовой итог», финальная строка минимальной длины тура.

```
=====
Входные рёбра (как записаны во входном файле)
=====
+-----+-----+-----+
| Номер | u | v | Длина |
+-----+-----+-----+
| 1 | 1 | 2 | 3 |
| 2 | 2 | 3 | 100 |
| 3 | 3 | 4 | 10 |
| 4 | 4 | 5 | 1 |
+-----+-----+-----+

----- Матрица кратчайших расстояний после Флойда-Уоршелла -----

+-----+-----+-----+-----+-----+
| Из \ В | Г1 | Г2 | Г3 | Г4 | Г5 |
+-----+-----+-----+-----+-----+
```

|                                            |    |  |     |  |     |  |     |  |     |          |     |  |
|--------------------------------------------|----|--|-----|--|-----|--|-----|--|-----|----------|-----|--|
|                                            | Г1 |  | 0   |  | 3   |  | 103 |  | 113 |          | 114 |  |
|                                            | Г2 |  | 3   |  | 0   |  | 100 |  | 110 |          | 111 |  |
|                                            | Г3 |  | 103 |  | 100 |  | 0   |  | 10  |          | 11  |  |
|                                            | Г4 |  | 113 |  | 110 |  | 10  |  | 0   |          | 1   |  |
|                                            | Г5 |  | 114 |  | 111 |  | 11  |  | 1   |          | 0   |  |
| +-----+-----+-----+-----+-----+            |    |  |     |  |     |  |     |  |     |          |     |  |
| =====                                      |    |  |     |  |     |  |     |  |     |          |     |  |
| Числовой итог                              |    |  |     |  |     |  |     |  |     |          |     |  |
| =====                                      |    |  |     |  |     |  |     |  |     |          |     |  |
| +-----+-----+                              |    |  |     |  |     |  |     |  |     |          |     |  |
| Показатель                                 |    |  |     |  |     |  |     |  |     | Значение |     |  |
| +-----+-----+                              |    |  |     |  |     |  |     |  |     |          |     |  |
| Доп. длина дублируемых проездов (минимум)  |    |  |     |  |     |  |     |  |     | 114      |     |  |
| Базовая сумма рёбер                        |    |  |     |  |     |  |     |  |     | 114      |     |  |
| Итоговая минимальная длина                 |    |  |     |  |     |  |     |  |     | 228      |     |  |
| +-----+-----+                              |    |  |     |  |     |  |     |  |     |          |     |  |
| Минимальная длина маршрута почтальона: 228 |    |  |     |  |     |  |     |  |     |          |     |  |

**Комментарий к выводу на этом файле.** Табличная секция воспроизводит входные числа строка за строкой. Матрица Флойда для пяти городов: диагональ нулевая, прочие клетки — суммы минимальных путей между парами городов цепью. Последний числовой блок даёт минимально добавляемые дубликаты между нечётными, сумму простого покрытия каждого ребра входа один раз (**базовая**) и итог — длина маршрута китайского почтальёна.

## 9. Ссылка на исходный код

Исходный код задачи доступен по ссылке:

<https://github.com/mitya-y/formal-theories-tp-3-course/tree/master/Почтальон>